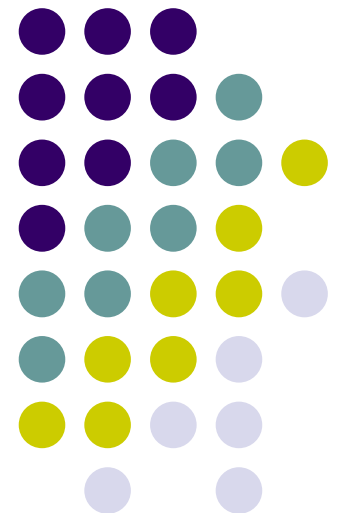
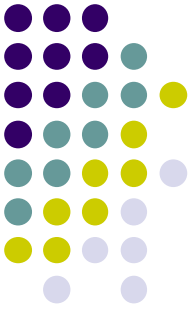


Introduction to Computer Communication

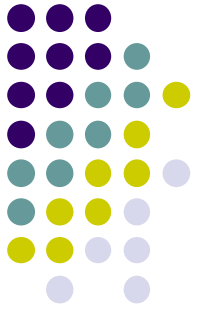
Boaz Patt-Shamir





Overview: Handout

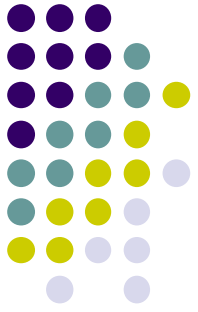
- Administrivia
- Course objective
- Homework, grading
- Bibliography
- Tentative schedule



Today: First Principles

- Requirements and building blocks
- Networks issues: switching, scheduling
- Network performance
- Layering and protocols
- Network layer model

Basic motivation

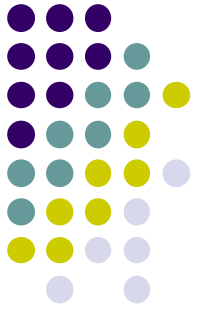


Goal: transfer information across space

- Question: what is information?
- The digital answer (Shannon):
w.l.o.g., string of bits.
- Requires:
Interpretation convention
 - Such a convention is a (low-level) protocol.

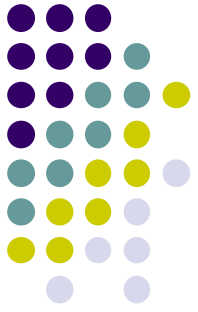


Network Overview

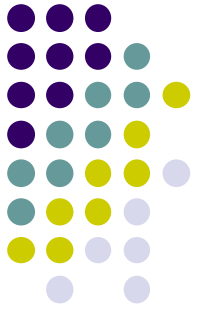


- What's a network?
 - Provide Connectivity: all can reach all, quickly, moving much information
 - Cheaply: Low cost by effective sharing
- How are networks designed and built?
 - Layering
 - Protocols
 - Standards

Requirements from networks



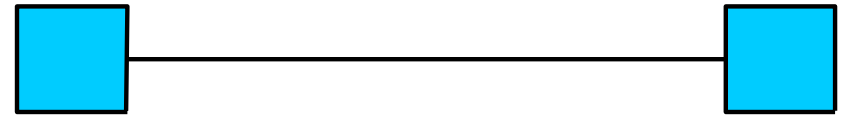
- Users: Services that their applications need
 - Guarantee that each message sent will be delivered without error within a certain amount of time
- Designers: Cost-effective design
 - Network resources are efficiently utilized and fairly allocated to different users
- Providers: System that is easy to administer and manage
 - Faults can be easily isolated and it is easy to account for usage



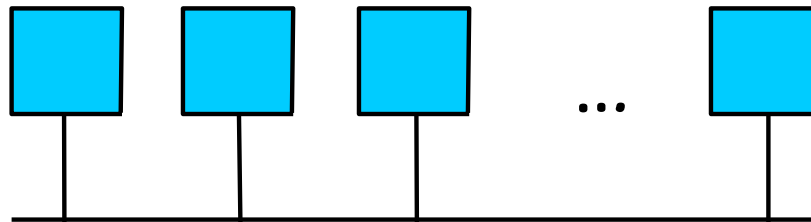
Building Blocks

- Nodes: workstations, routers, cellphones, tablets...
- Links: coax cable, optical fiber, radio waves...

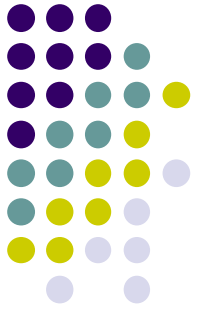
- Point-to-point



- Multiple access

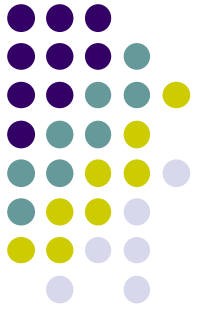


Sharing a multiple-access link

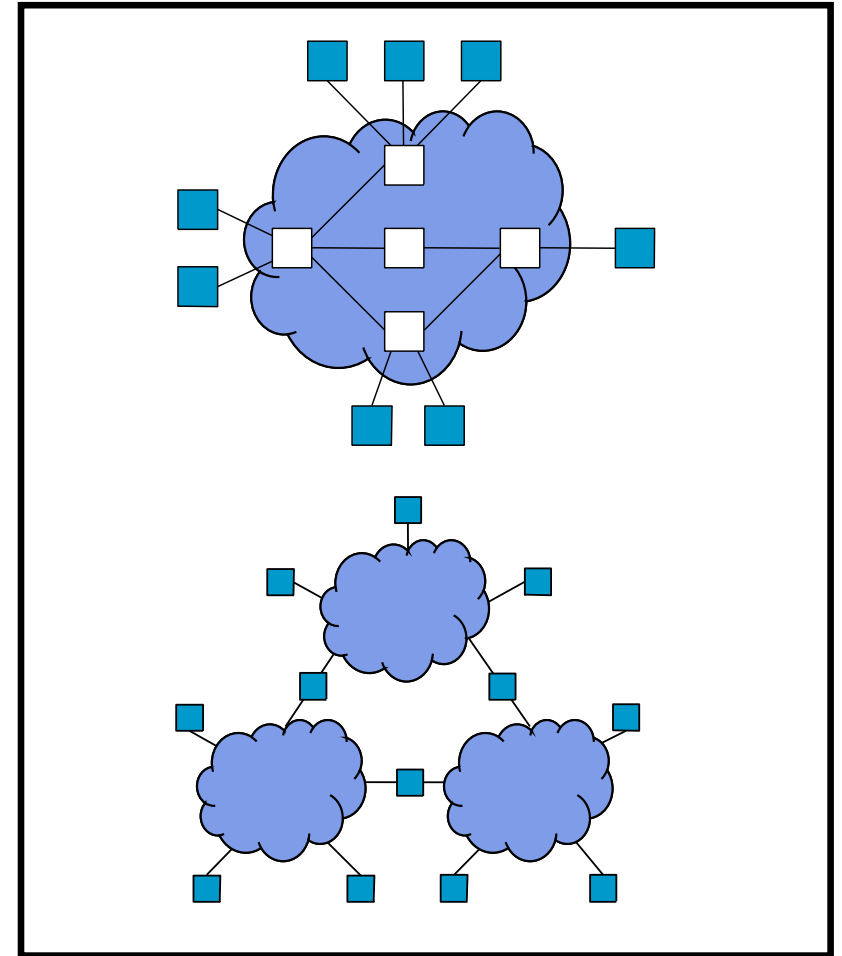


- On-demand time-division multiplexing
- Scheduling on a per-packet basis
- Packets from different sources are interleaved
- Statistical Multiplexing: Uses upper bounds to limit transmission
 - Can use queue size to determine share per source

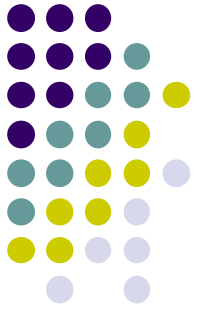
Indirect Connectivity



- Switched Networks
- Internetworks



Effects of Indirect Connectivity



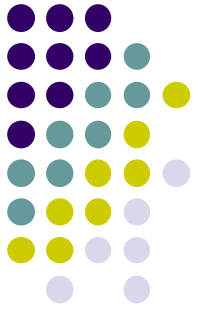
- Nodes receive data on one link and forward it onto the next → switching network
 - Circuit Switching
 - Telephone
 - Stream-based (dedicated circuit)
 - Packet Switching
 - Internet
 - Message-based (store-and-forward)

Switching Strategies



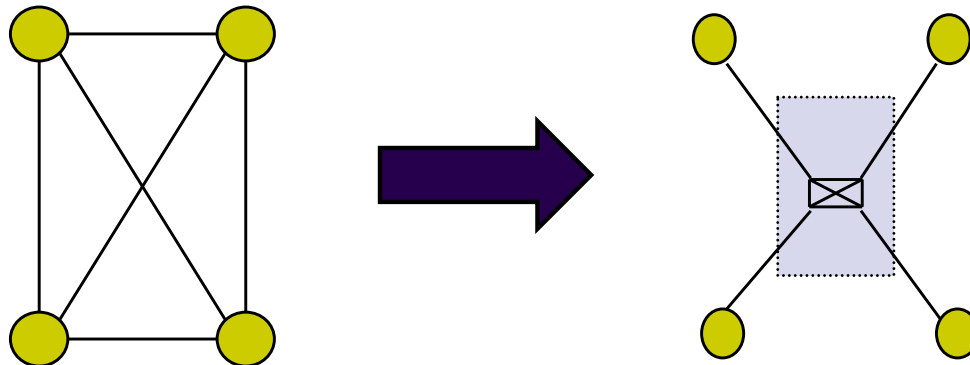
- Circuit switching
 - Dedicated communication circuit (e.g., telephone)
 - Links reserved for use by communication channel
 - Constant rate bit stream
- Packet switching
 - Links used dynamically
 - Send/receive messages (packets)
 - Admission policies and other traffic determine bandwidth



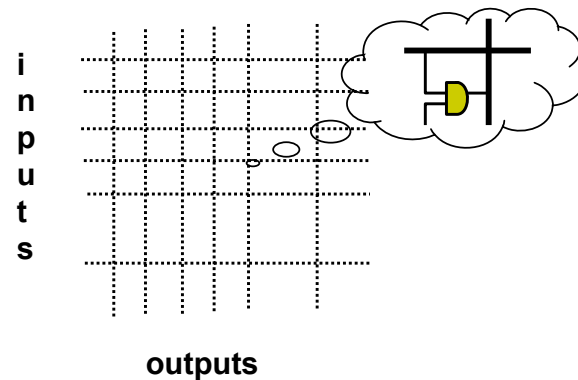


Switch structure

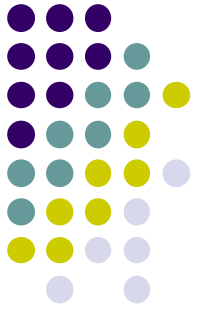
- Idea: contract a network into a box



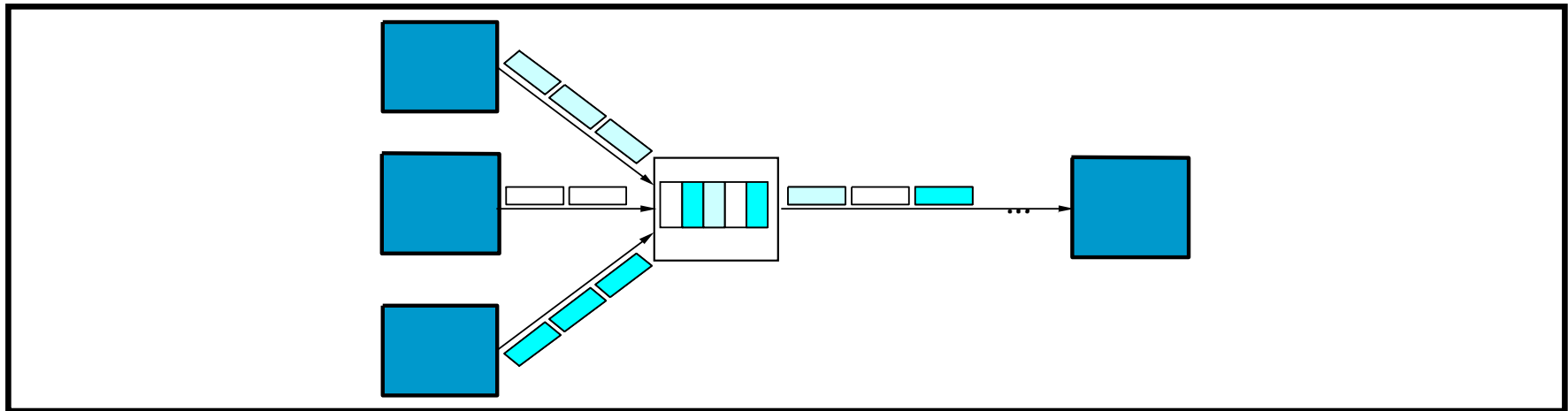
- Allows for interesting designs



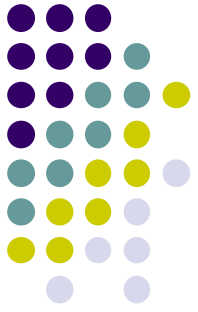
Scheduling in a Switch



- Each switch implements a scheduling policy
- Issues:
 - How do we make these decisions fairly? Round Robin? FIFO?
 - How should the switch handle congestion?

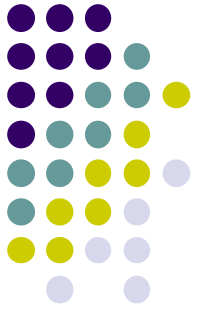


Functionality

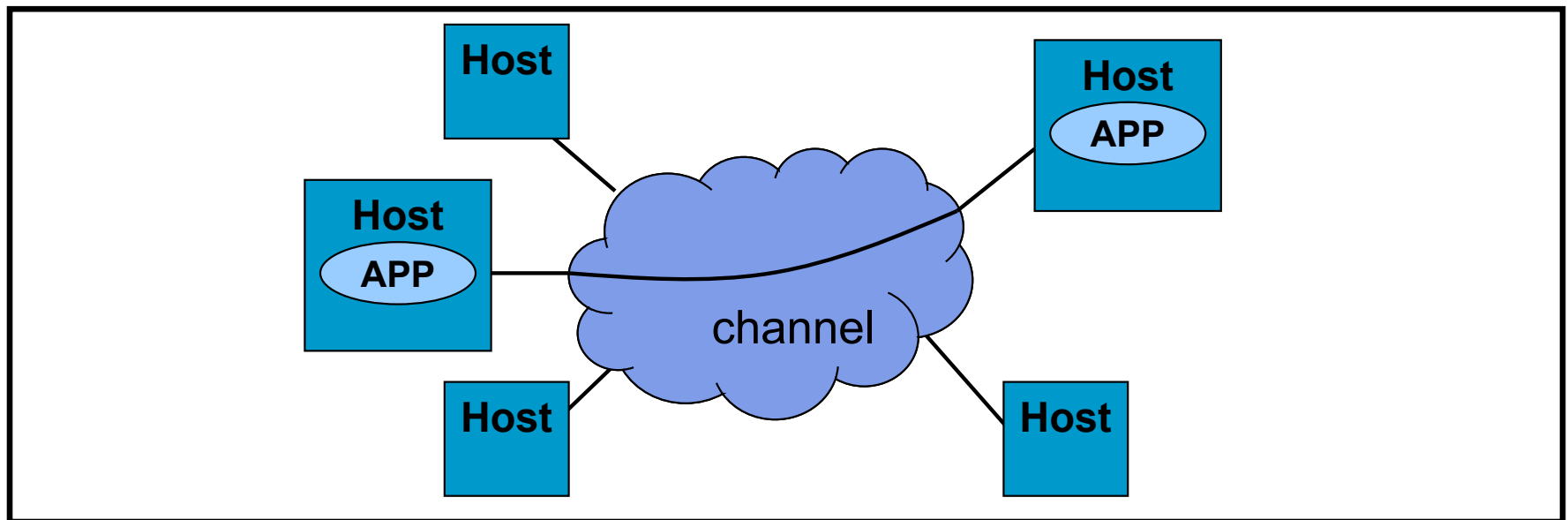


- Support for common services.
 - Goal: Meaningful communication between hosts on a network
 - Idea:
 - Common services simplify the role of applications
 - Hide the complexity of the network without overly constraining the application designer

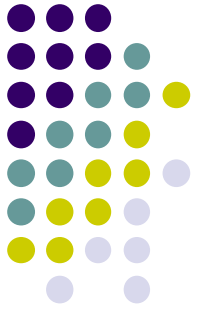
Example: Channels



- Channel: An abstraction for application-level communication
- Compare with a circuit in a telephone network

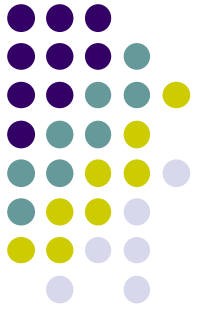


Channel Implementation



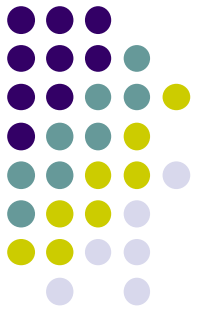
- Issue:
 - Where does the functionality belong?
 - In the network core (switches)?
 - That's the way in telephone systems
 - In the network edges (end hosts)?
 - That's the original idea of the Internet

Reliability



- Goal: Hide failures
 - Make the network appear more reliable: use redundancy
- What goes wrong in the network?
 - Bits are corrupted in transit
 - Links break down
 - Processors crash
 - Hostile intruders

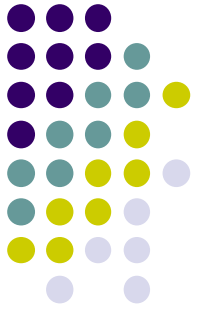
Performance Metrics: Length and Width



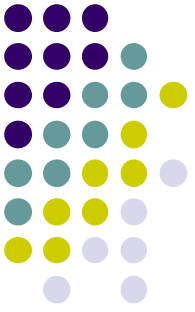
- Latency/Delay: how long it takes for the first bit to get to the other side
 - seconds
- Bandwidth: how long it takes to push a message into the link
 - Bits per second (bps) (e.g., 10Mbps Ethernet)
 - Note: Mbps = 10^6 bits per second, but MB = 2^{20} bytes



Latency/Delay

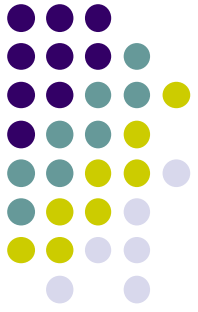


- Three components:
 - Latency = Propagation + Transmit + Queue
 - Propagation Delay: distance / speed of light
 - Transmission Delay: data size / network bandwidth
 - Queuing Delays: traffic and processing in switches
 - Software processing overheads
- RTT - Round Trip Time
 - Time for a message from A to B and Back to A
 - An important factor for pipelining



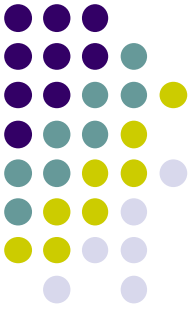
Protocols

- A *protocol* is a set of rules and formats that govern the communication between communicating **peers**
 - set of valid messages
 - meaning of each message
- A protocol is necessary for any function that requires cooperation between peers



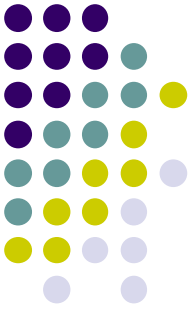
Example: Web Browsing

- HTTP server, client = browser
 - Client sends a string of requests to server
 - Server sends back a stream of responses
 - Sending over TCP sockets
- What is a request, a response?
- How to make sure that communication goes through?
- Responses can come back out of order
- How to end the dialog?



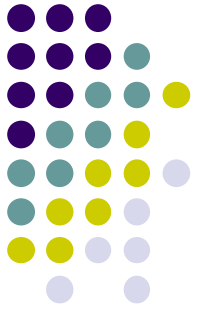
What's a protocol?

- Syntax of an event
 - How is its occurrence defined?
 - For messages: in what format?
- Actions to take on the occurrence of an event
 - for example, send messages back, make an output to application
 - Implicitly defines *meaning* of the event



Protocols: Another view

- As providing a *service*
- For example, SCP provides *secure, reliable file transfer service*
- Peer entities use a protocol to provide a service to a higher-level peer entity



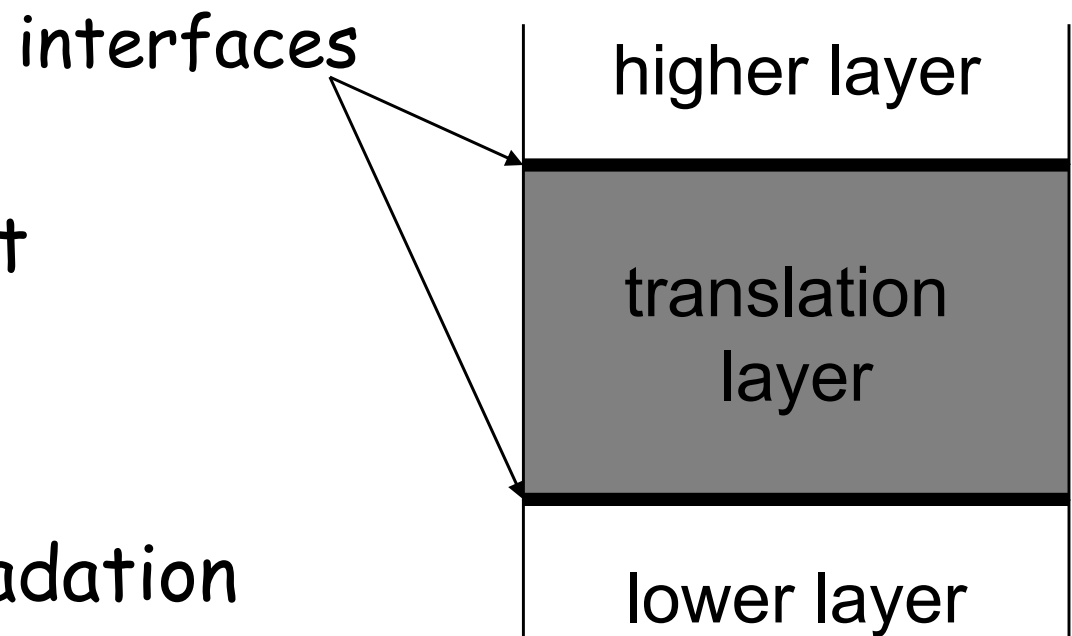
Engineering in Layers

Advantages:

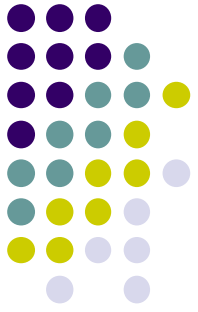
- Modularity
- Reuse
- Easier development

Disadvantage:

- Performance degradation

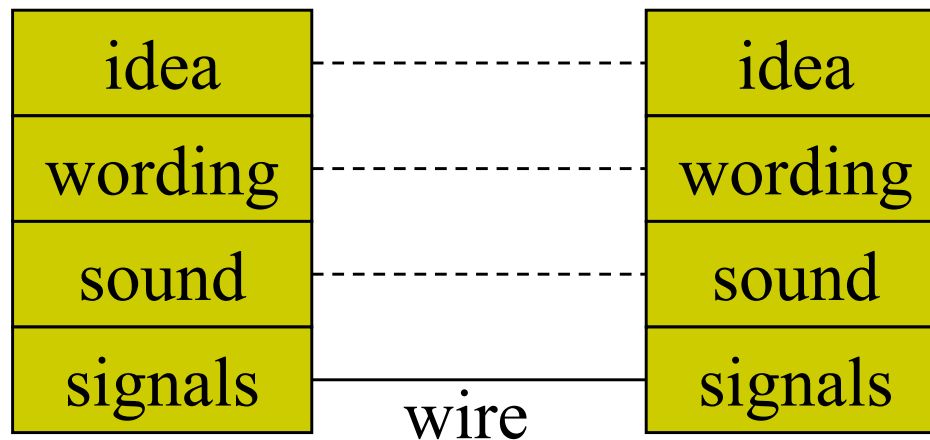


Idea: each interface is a **language** supported by the lower layer, used by the upper layer; each layer just **translates**

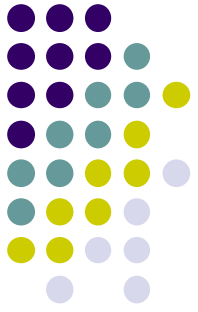


The Layering Approach

A telephone call:



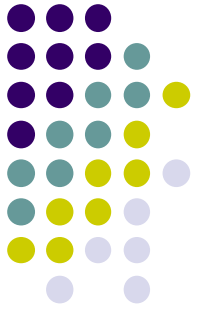
- Each layer obeys strict interface rules
- Each layer “talks” to its counterpart only



Protocol stack

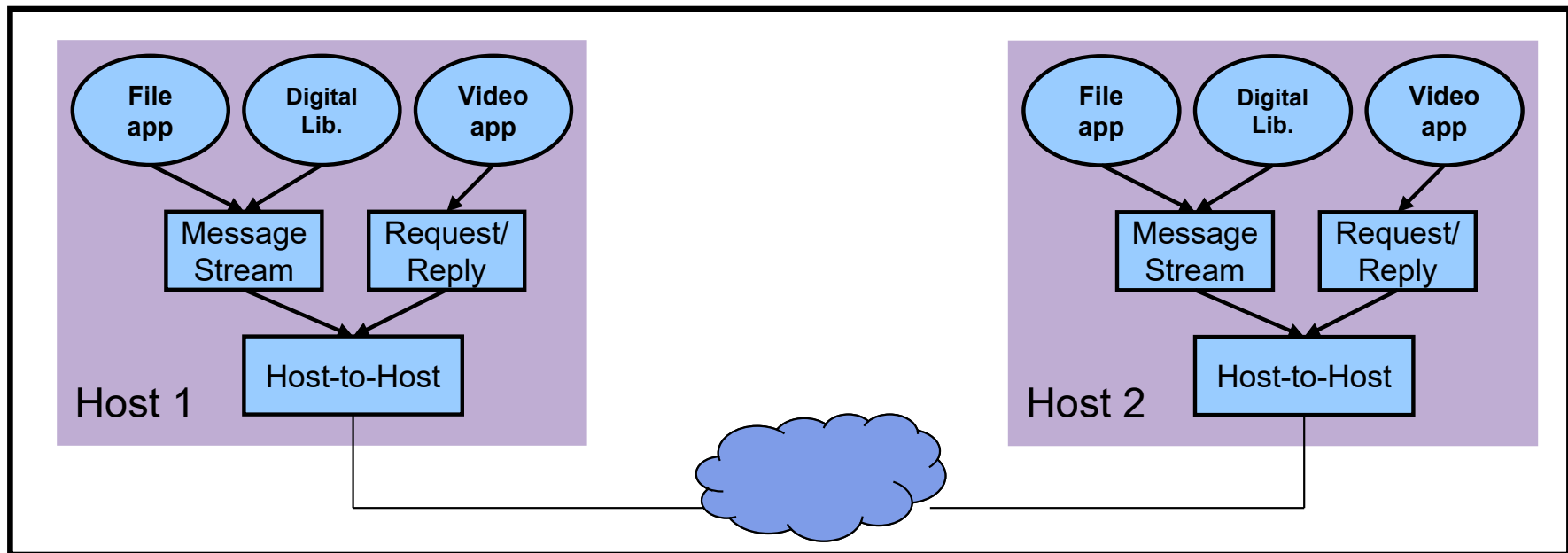
- A set of protocol layers
- Each layer uses the layer below and provides a service to the layer above
- Key idea
 - once we define a service provided by a layer, we need know nothing more about the details of how the layer actually implements the service
 - information hiding
 - decouples changes

Protocol Machinery

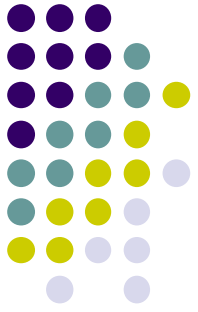


- Protocol Graph

- most peer-to-peer communication is indirect
- peer-to-peer is direct only at hardware level
- dependency relationship through service interface

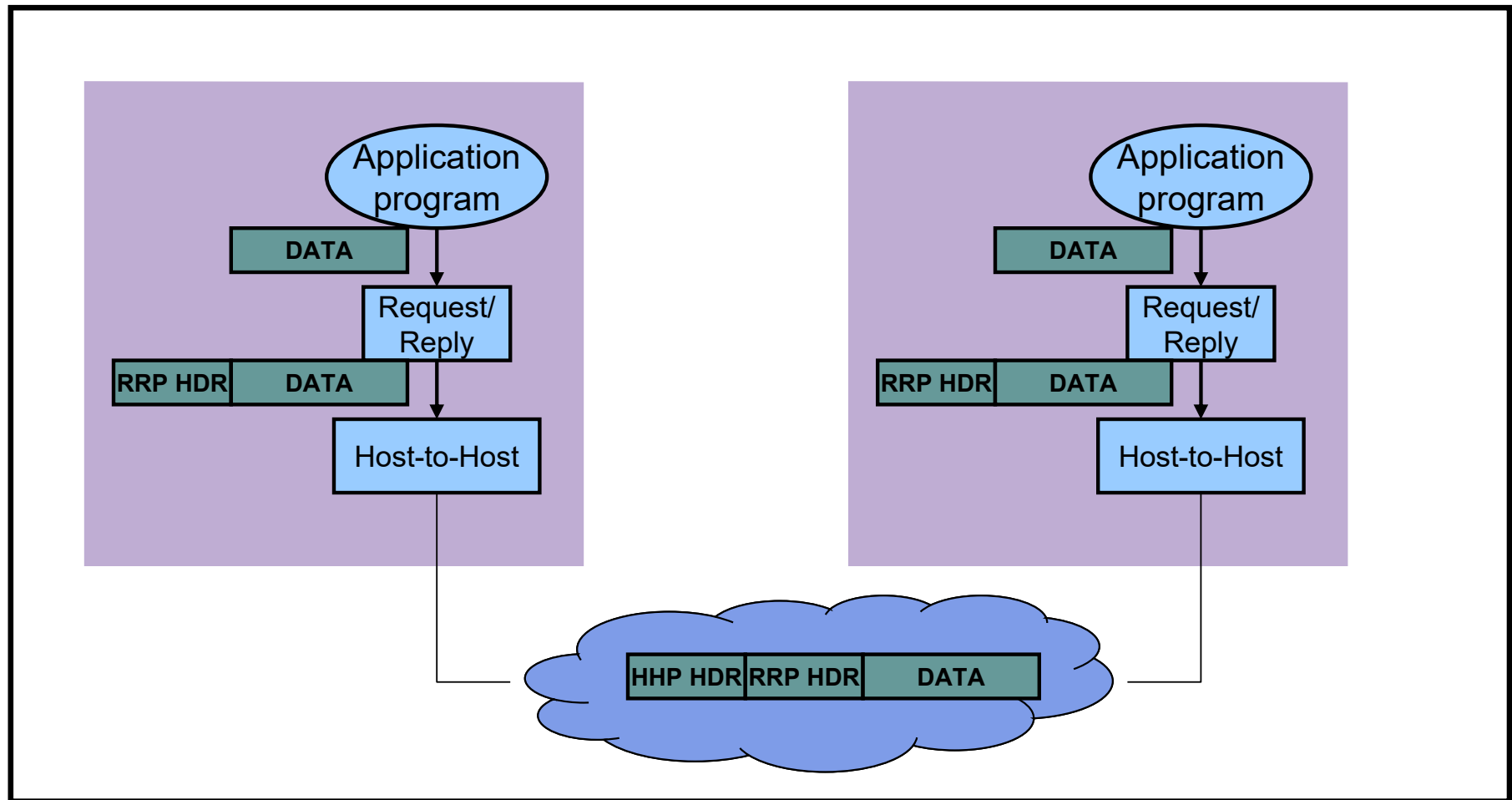
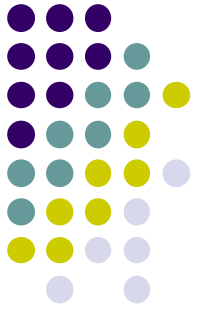


Layering Concepts

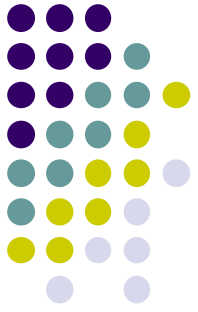


- Encapsulation
 - Higher layer protocols create messages and send them via the lower layer protocols
 - These messages are treated as data by the lower-level protocol
 - Higher-layer protocol adds its own control information in the form of headers or trailers
- Multiplexing and Demultiplexing
 - Use protocol keys in the header to determine correct upper-layer protocol

Encapsulation

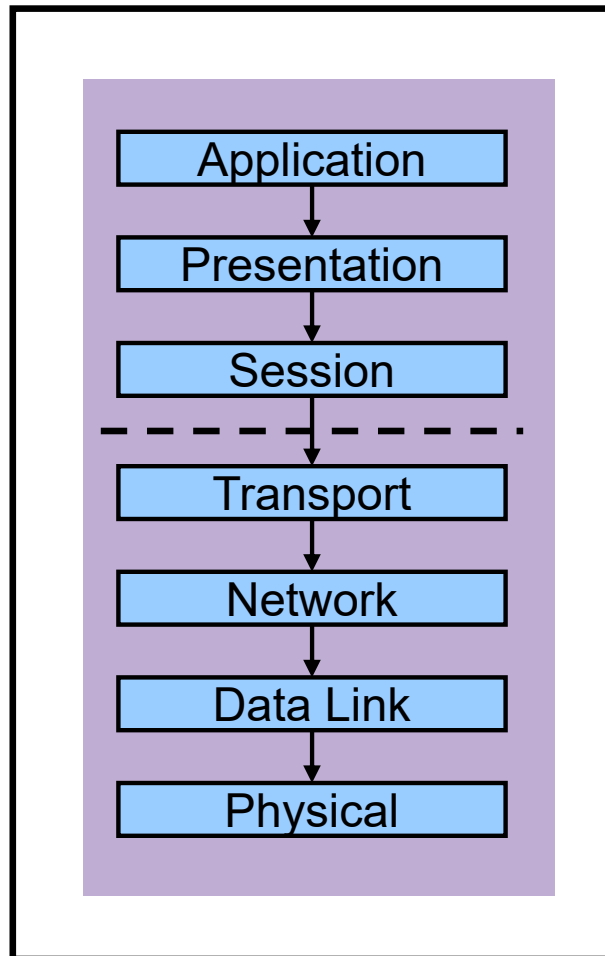
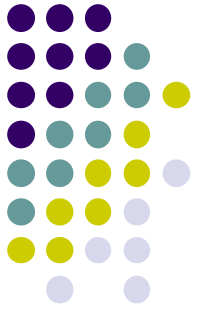


OSI Architecture



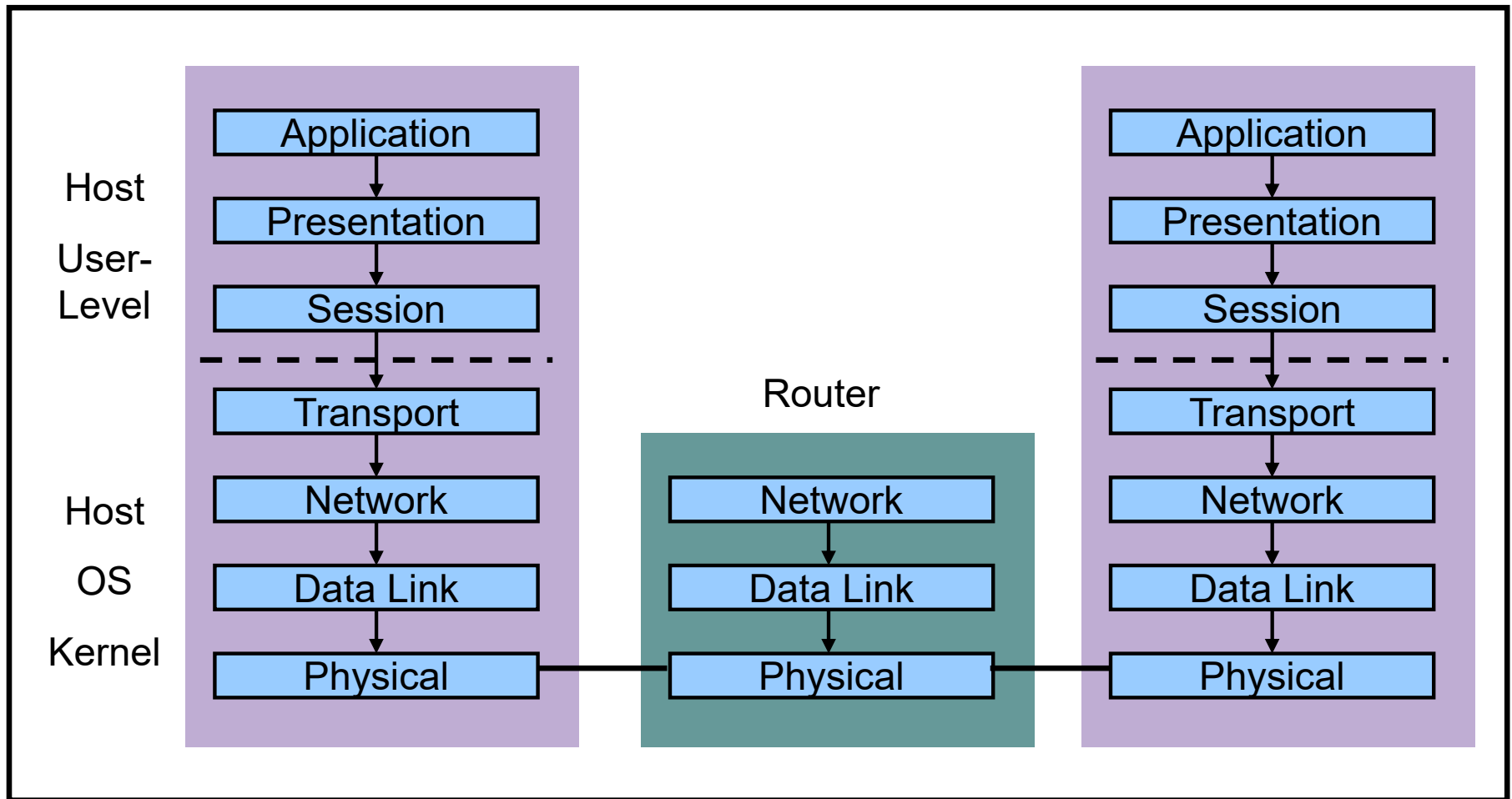
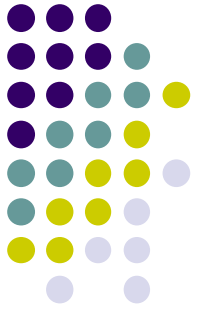
- Open Systems Interconnect (OSI) Architecture
 - International Standards Organization (ISO)
 - International Telecommunications Union (ITU, formerly CCITT)
 - "X dot" series: X.25, X.400, X.500
 - Primarily a reference model

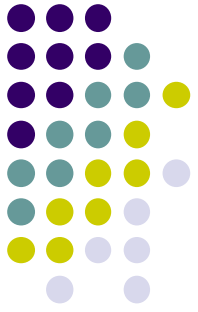
OSI Protocol Stack



- Application: Application specific protocols
- Presentation: Format of exchanged data
- Session: Name space for connection mgmt
- Transport: process-to-process channel
- Network: Host-to-host packet delivery
- Data Link: Framing of data bits
- Physical: transmission of raw bits

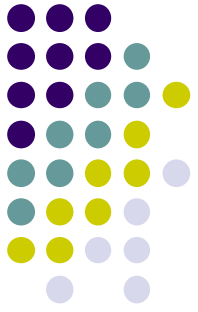
OSI Protocol Stack





Example: Postal network

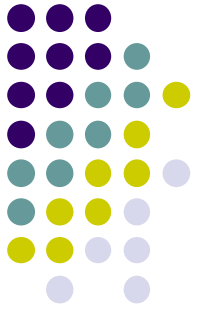
- Application: send bill
- Session and presentation: chief clerk (system) decides: courier/mail/email. Maybe choose language.
- Transport: mail clerk drops mail at post office
 - Like sending a message, retransmit if not unsuccessful
- Network: Postal system computes a route and forwards the letters
- Datalink layer: letters carried by planes, trains, trucks, bikes
- physical layer: the letter itself



Physical layer

Service provided: **Move bits between physically connected end-systems**

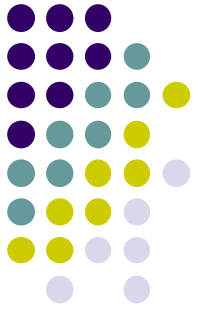
- Standard prescribes
 - coding scheme to represent a bit
 - shapes and sizes of connectors
 - bit-level synchronization
- Internet
 - technology to move bits on a wire, wireless link, satellite channel etc.



Datalink layer

Service provided: **Reliable communication over a single link.**

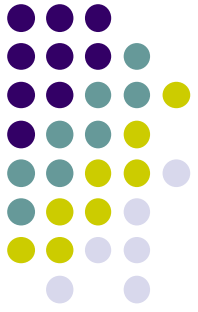
- Introduces the notion of a *frame*: a set of bits that belong together
 - *Begin* and *end* markers delimit a frame
- May be broadcast links with more than two endpoints! (ethernet, wireless)
 - need datalink-layer address
 - also need to decide who gets to speak next
 - Provided by Medium Access Control sublayer (*MAC*) sublayer
- Internet: a variety of datalink layer protocols
 - most common is Ethernet
 - others are wireless ethernet, bluetooth, FDDI, SONET...



Network layer

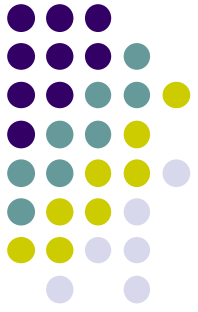
Service provided: **Carry data from source to destination.**

- Allows an end-system to communicate with any other end-system by computing a route between them
- Logically concatenates a set of links to form the abstraction of an **end-to-end** link
- Hides idiosyncrasies of datalink layer
- Provides unique network-wide addresses
- Found both in end-systems and in intermediate systems



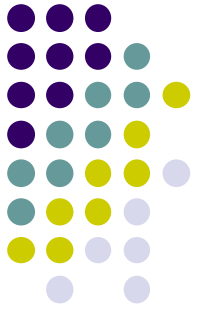
Network layer types

- Packet switching networks:
 - provides both routing and data forwarding
- Circuit switching networks:
 - separate data plane and control plane
 - data plane only forwards and schedules data (touches every byte)
 - control plane responsible for routing, call-establishment, call-teardown (doesn't touch data bytes)
- Internet network layer protocol: IP
 - packet-forwarding, routing, scheduling, unique IP addresses
 - found in all end-systems and intermediate systems
 - can be layered over anything, but only best-effort service



Transport layer

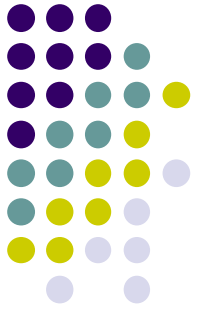
- **Reliable end-to-end communication.**
- creates the abstraction of an *error-controlled, flow-controlled and multiplexed* end-to-end link (Network layer provides only a 'raw' end-to-end service)
- Some transport layers provide fewer services
 - e.g. simple error detection, no flow control, and no retransmission
- Internet
 - TCP provides error control, flow control, multiplexing
 - UDP provides only multiplexing



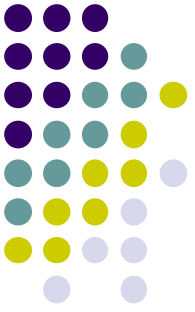
Transport layer (contd.)

- Error control: messages should reach destination despite packet loss, corruption and duplication
 - How: retransmit lost packets; detect, discard, and retransmit corrupted packets; detect and discard duplicated packets
- Flow control: match transmission rate to rate currently sustainable all the way to (and including) the destination.
- Multiplexing: isolate multiple applications to the same end-to-end connection
 - How: tagging. Internet: add an application-specific identifier (*port number*) so that receiving end-system can hand in incoming packet to the correct application

Session layer



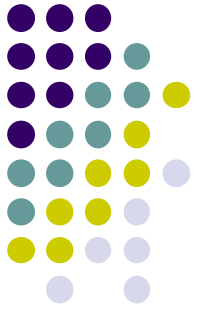
- Provides *full-duplex service, expedited data delivery, and session synchronization*
- Duplex
 - if transport layer is simplex, concatenates two transport endpoints together
- Expedited data delivery
 - allows some messages to skip ahead in end-system queues, by using a separate low-delay transport layer endpoint
- Synchronization
 - allows users to place marks in data stream and to roll back to a pre-specified mark



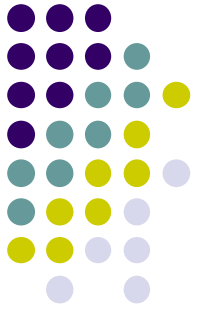
Presentation layer

- Usually *ad hoc*
- Touches the application data
(Unlike other layers which deal with *headers*)
- Hides data representation differences between applications
 - characters (*ASCII, unicode, EBCDIC.*)
- Can also encrypt data
- Internet
 - no standard presentation layer
 - only defines network byte order for 2- and 4-byte integers

Application layer



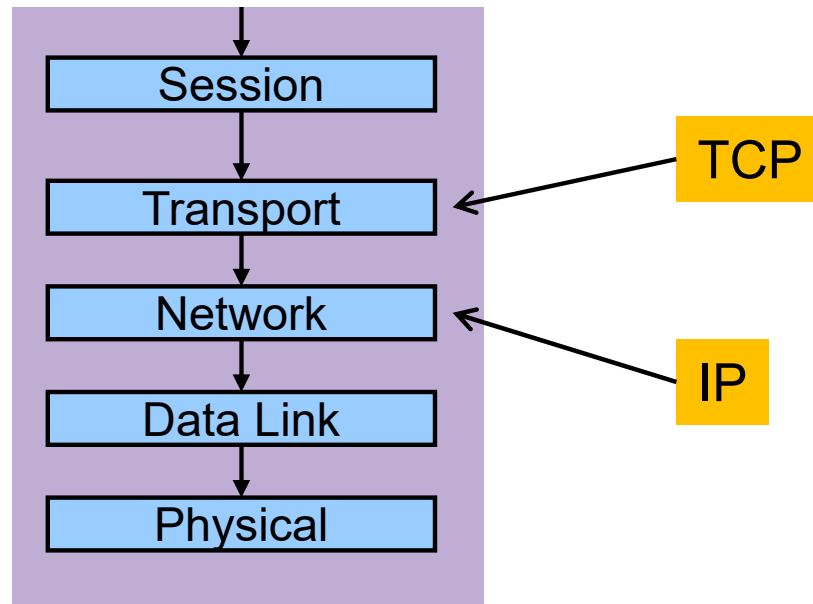
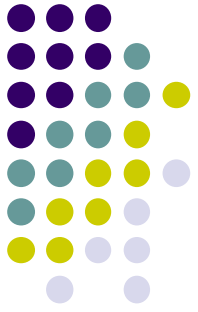
- The set of applications that use the network
- Doesn't provide services to any other layer



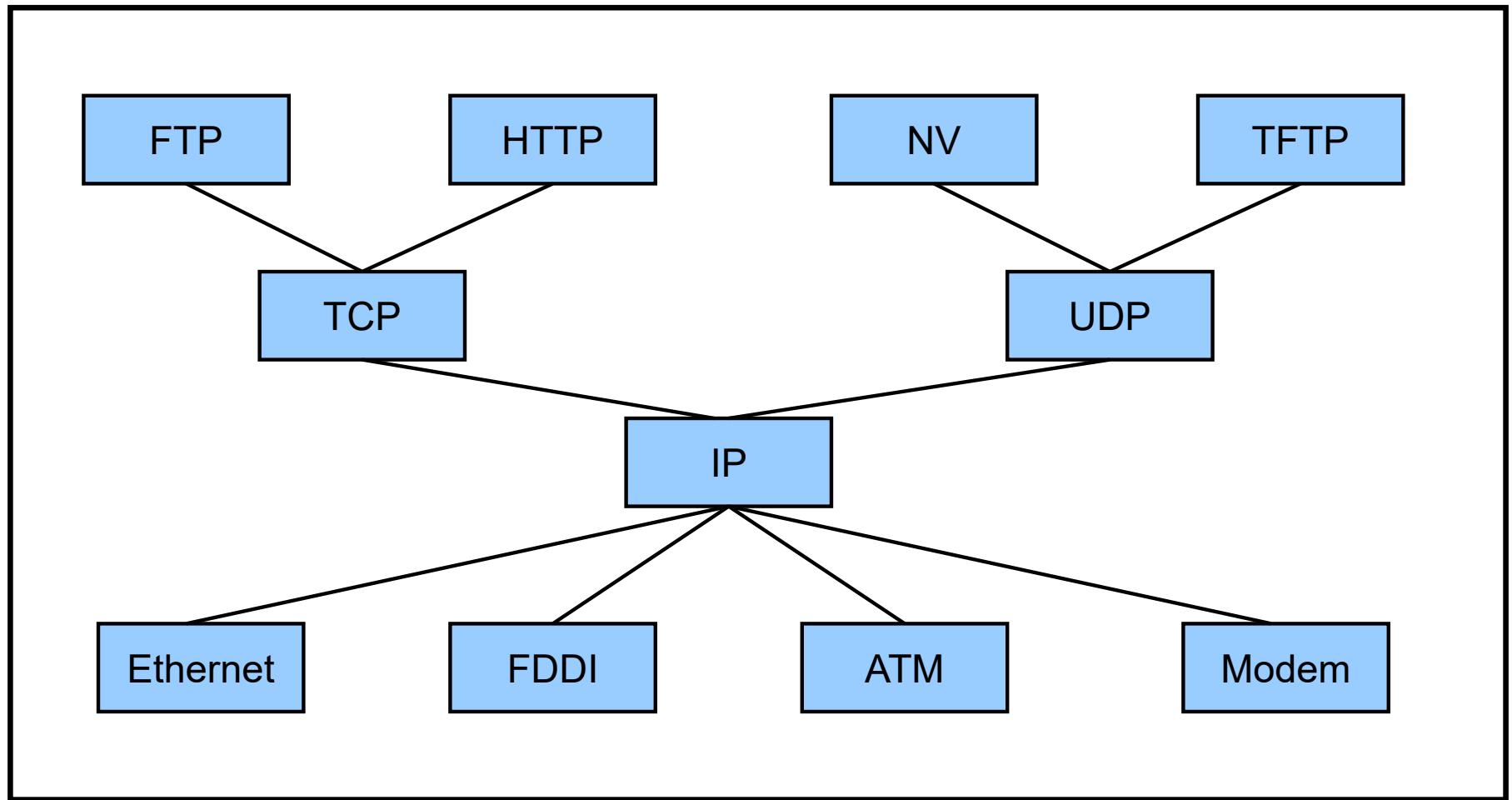
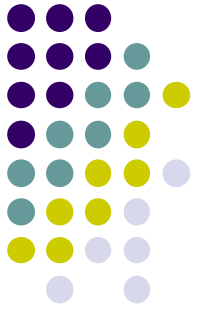
Discussion

- Layers break a complex problem into smaller, simpler pieces.
 - But pay a price in performance
- Why seven layers?
 - Need a top and a bottom → 2
 - Need to hide physical link; so need datalink → 3
 - Need both end-to-end and hop-by-hop actions; so need at least the network and transport layers → 5

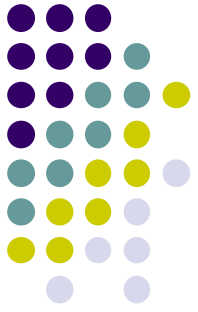
Internet: 5 layer model



Internet Architecture - Hourglass Design



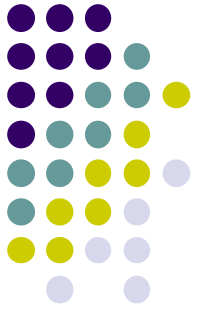
Internet Architecture (TCP/IP)



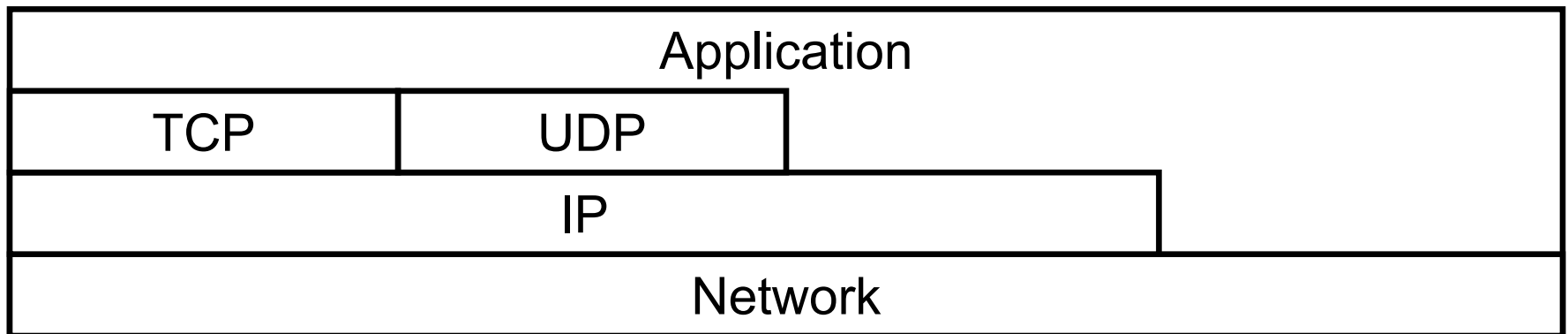
- Originated by ARPA, later NSF. Popularized with release of BSD Unix; *i.e.*, free software
- Internet Engineering Task Force (IETF)
 - Culture: implement, then standardize
 - OSI culture: standardize, then implement
 - Standard suggestions debated publicly through "requests for comments" (RFC's)

We reject kings, presidents, and voting.
We believe in rough consensus and running code.
- David Clark

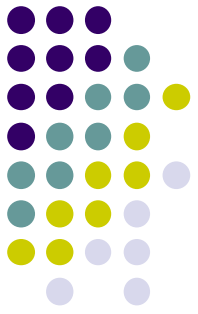
Internet Architecture



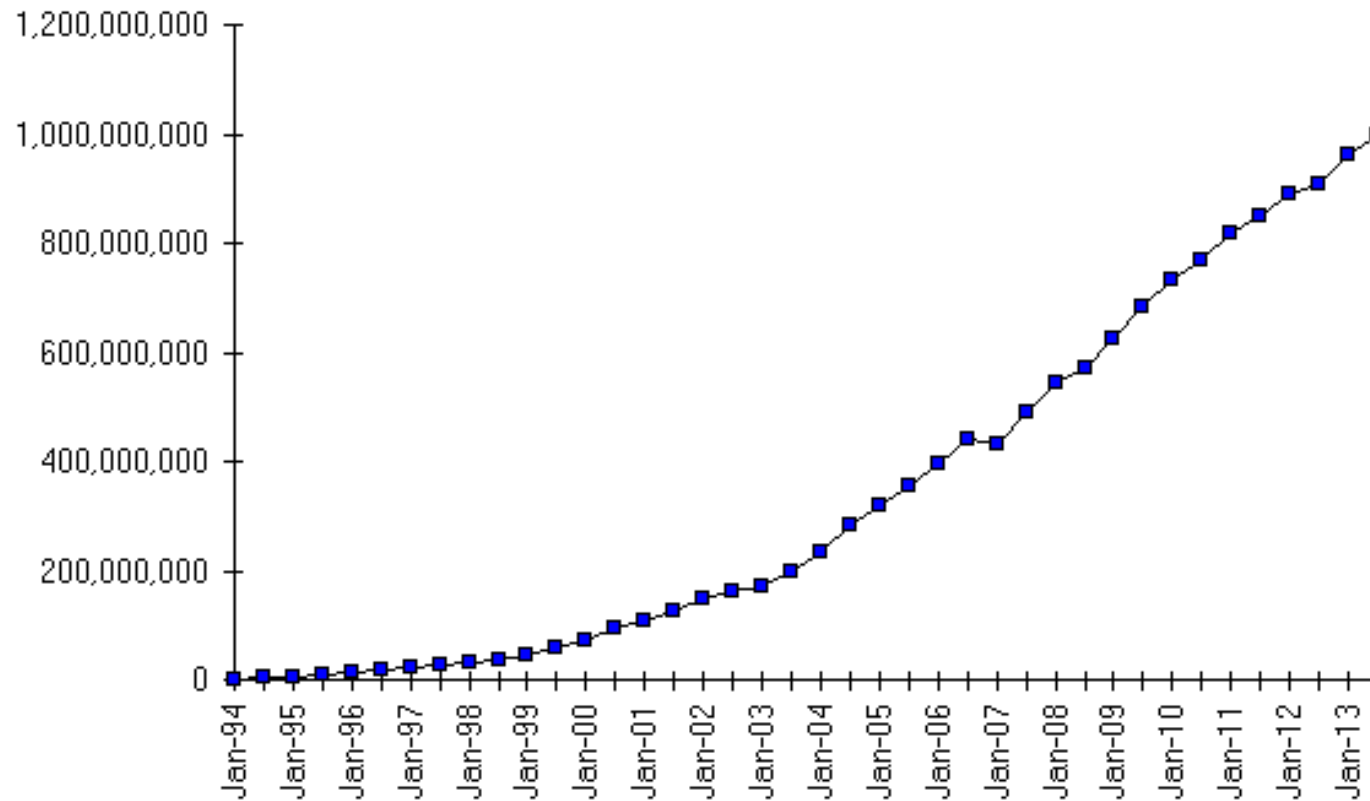
- Features:
 - No strict layering
 - Hourglass shape – IP is the focal point



The Internet Boom

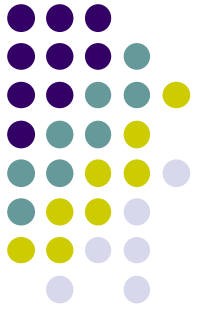


Internet Domain Survey Host Count

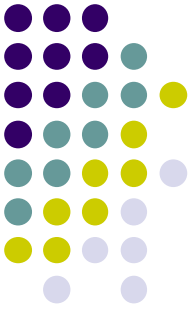


Source: Internet Systems Consortium (www.isc.org)

Protocol Acronyms



- (T)FTP - (Trivial) File Transfer Protocol
- HTTP - HyperText Transport Protocol
- SMTP - Simple Mail Transfer Protocol
- NTP - Network Time Protocol
- TCP - Transmission Control Protocol
- UDP - User Datagram Protocol
- IP - Internet Protocol
- FDDI - Fiber Distributed Data Interface
- ATM - Asynchronous Transfer Mode



New Concepts

- Latency
- Bandwidth
- Protocol
- Software interface
- Protocol stack
- Encapsulation
- Physical layer
- Datalink layer
- Network layer
- Transport layer
- Application layer